

Guía de Documentación de Código PHP con PHPDoc

¿Qué es PHPDoc?

PHPDoc es un sistema de documentación basado en comentarios que sigue el estándar JavaDoc. Permite describir clases, métodos, propiedades, parámetros y tipos de retorno de manera estructurada.

Sintaxis Básica

Estructura de un Bloque PHPDoc

```
/**
 * Descripción breve de una línea
 *
 * Descripción detallada opcional que puede extenderse
 * en múltiples líneas para explicar el comportamiento
 * y detalles de implementación.
 *
 * @tag Valor de la etiqueta
 */
```

Reglas importantes: - Comienza con `/**` y termina con `*/` - Cada línea intermedia comienza con `*` - Se coloca inmediatamente antes del elemento a documentar - La primera línea es un resumen breve - Después de una línea en blanco, viene la descripción detallada - Las etiquetas (tags) van al final, precedidas por `@`

Etiquetas PHPDoc Principales

Etiquetas de Descripción

- **@param** - Documenta parámetros de funciones/métodos
- **@return** - Documenta el valor de retorno
- **@var** - Documenta variables y propiedades
- **@throws** - Documenta excepciones que puede lanzar

Etiquetas de Información

- **@author** - Autor del código
- **@version** - Versión del elemento
- **@since** - Versión desde la que existe
- **@deprecated** - Marca código obsoleto

- **@see** - Referencia a otros elementos
- **@link** - Enlace a documentación externa

Etiquetas de Herencia y Relaciones

- **@inheritDoc** - Hereda documentación del padre
- **@uses** - Indica qué elementos utiliza
- **@property** - Documenta propiedades mágicas
- **@method** - Documenta métodos mágicos

Tipos de Datos en PHPDoc

Tipos Simples

- **string** - Cadena de texto
- **int** o **integer** - Número entero
- **float** - Número decimal
- **bool** o **boolean** - Booleano
- **array** - Array
- **object** - Objeto genérico
- **resource** - Recurso
- **null** - Valor nulo
- **mixed** - Cualquier tipo
- **void** - Sin retorno (solo para @return)

Tipos Compuestos

```
// Array de strings
@param string[] $nombres

// Array asociativo
@param array<string, int> $edades

// Múltiples tipos posibles
@param string|int $identificador

// Tipo nullable (puede ser null)
@param ?string $descripcion
@param string|null $descripcion

// Callable
@param callable $callback

// Clase específica
@param Usuario $usuario
```

Ejemplos Prácticos

Documentar una Clase

```
<?php

/**
 * Gestiona las operaciones de usuarios en la aplicación
 *
 * Esta clase proporciona métodos para crear, leer, actualizar
 * y eliminar usuarios de la base de datos, además de manejar
 * la autenticación y autorización.
 *
 * @package App\Models
 * @author Juan Pérez <juan@ejemplo.com>
 * @version 1.2.0
 * @since 1.0.0
 */
class GestorUsuarios
{
    // Contenido de la clase
}
```

Documentar Propiedades

```
/**
 * Conexión a la base de datos
 *
 * @var PDO
 */
private $conexion;

/**
 * Lista de usuarios cargados
 *
 * @var Usuario[]
 */
private $usuarios = [];

/**
 * Configuración del gestor
 *
 * @var array<string, mixed>
 */
protected $config;
```

Documentar Constructores

```
/**
 * Constructor del gestor de usuarios
 *
 * Inicializa la conexión a la base de datos y carga
 * la configuración necesaria para el funcionamiento.
 *
 * @param PDO $conexion Instancia de conexión PDO
 * @param array $config Configuración del gestor
 * @throws InvalidArgumentException Si la configuración es inválida
 */
public function __construct(PDO $conexion, array $config = [])
{
    $this->conexion = $conexion;
    $this->config = $config;
}
```

Documentar Métodos

```
/**
 * Crea un nuevo usuario en la base de datos
 *
 * Valida los datos del usuario, hashea la contraseña
 * y almacena el registro en la tabla de usuarios.
 *
 * @param string $nombre Nombre completo del usuario
 * @param string $email Correo electrónico (debe ser único)
 * @param string $password Contraseña en texto plano (será hasheada)
 * @return Usuario Objeto del usuario creado con su ID asignado
 * @throws DatabaseException Si falla la inserción en la base de datos
 * @throws ValidationException Si los datos no son válidos
 * @see validarDatosUsuario()
 */
public function crearUsuario(string $nombre, string $email, string $password): Usuario
{
    // Implementación
}
```

Documentar Métodos con Opciones

```
/**
 * Busca usuarios según criterios específicos
 *
 * @param array $filtros Criterios de búsqueda
 *      - 'nombre' (string): Buscar por nombre
 *      - 'email' (string): Buscar por email
 */
```

```

*           - 'activo' (bool): Filtrar por estado activo
* @param int $limite Número máximo de resultados (por defecto: 10)
* @param int $offset Desplazamiento para paginación (por defecto: 0)
* @return Usuario[] Array de objetos Usuario encontrados
*/
public function buscarUsuarios(array $filtros, int $limite = 10, int $offset = 0): array
{
    // Implementación
}

```

Documentar Funciones Globales

```

/**
 * Formatea una cantidad monetaria a formato local
 *
 * @param float $cantidad Cantidad a formatear
 * @param string $moneda Código ISO de la moneda (por defecto: 'EUR')
 * @return string Cantidad formateada con símbolo de moneda
 */
function formatearMoneda(float $cantidad, string $moneda = 'EUR'): string
{
    // Implementación
}

```

Documentar Clases con Propiedades Mágicas

```

/**
 * Modelo base para entidades
 *
 * @property int $id Identificador único
 * @property string $createdAt Fecha de creación
 * @property string $updatedAt Fecha de última actualización
 * @method void save() Guarda los cambios en la base de datos
 * @method void delete() Elimina el registro
 */
abstract class ModeloBase
{
    // Implementación
}

```

Buenas Prácticas

1. Documenta Todo lo Público

Todas las clases, métodos y propiedades públicas deben estar documentados. El código privado es opcional pero recomendado.

2. Sé Claro y Conciso

- La descripción breve debe caber en una línea
- Usa lenguaje claro y directo
- Evita redundancias obvias

```
// Mal
/**
 * Este método retorna el nombre
 * @return string retorna el nombre
 */
public function getNombre(): string

// Bien
/**
 * Obtiene el nombre completo del usuario
 * @return string
 */
public function getNombre(): string
```

3. Especifica Tipos Siempre

Incluso cuando PHP permite type hints, PHPDoc añade información adicional:

```
/**
 * @param string[] $emails Array de direcciones de email
 * @return bool True si todos son válidos
 */
public function validarEmails(array $emails): bool
```

4. Documenta Excepciones

Lista todas las excepciones que puede lanzar un método:

```
/**
 * @throws InvalidArgumentException Si el email no es válido
 * @throws DatabaseException Si falla la conexión
 * @throws DuplicateException Si el email ya existe
 */
```

5. Usa @deprecated Apropriadamente

Cuando marques código como obsoleto, indica la alternativa:

```
/**
 * @deprecated desde 2.0.0, usar getUsuarioPorId() en su lugar
 */
public function obtenerUsuario(int $id): ?Usuario
```

6. Mantén la Documentación Actualizada

Cuando cambies código, actualiza la documentación inmediatamente.

7. Documenta Comportamientos No Obvios

```
/**
 * Calcula el precio con descuento
 *
 * Nota: Los descuentos se aplican después de impuestos
 * y están limitados al 50% del precio original.
 *
 * @param float $precio Precio base sin impuestos
 * @param float $descuento Porcentaje de descuento (0-100)
 * @return float Precio final con impuestos y descuento
 */
```

8. Agrupa Documentación Relacionada

```
/**
 * Configuración de conexión a la base de datos
 *
 * @var string $host Host del servidor
 * @var int $port Puerto de conexión
 * @var string $database Nombre de la base de datos
 */
```

Generación de Documentación

Herramientas Disponibles

1. phpDocumentor (Recomendado) Instalación - Opción A: PHAR (Sin dependencias):

```
# Descargar la versión PHAR
wget https://phpdoc.org/phpDocumentor.phar
# O con curl
curl -O https://phpdoc.org/phpDocumentor.phar

# Dar permisos de ejecución
chmod +x phpDocumentor.phar

# Mover a un directorio en el PATH (opcional)
sudo mv phpDocumentor.phar /usr/local/bin/phpdoc

# Verificar instalación
phpdoc --version
```

Instalación - Opción B: Con Composer (si lo usas):

```
# Globalmente
composer global require phpdocumentor/phpdocumentor

# O en el proyecto
composer require --dev phpdocumentor/phpdocumentor
```

Instalación - Opción C: Docker (si lo usas):

```
docker pull phpdoc/phpdoc
```

Instalación - Opción D: Global en Windows:

```
# Descargar phpDocumentor.phar a C:\php-tools\
# Crear un archivo phpdoc.bat:
@echo off
php "C:\php-tools\phpDocumentor.phar" %*
```

```
# Agregar C:\php-tools\ al PATH del sistema
```

Uso básico:

```
# Con PHAR o instalación global
phpdoc -d ./src -t ./docs

# Con Composer local
php vendor/bin/phpdoc -d ./src -t ./docs

# Con Docker
docker run --rm -v $(pwd):/data phpdoc/phpdoc -d /data/src -t /data/docs

# Con más opciones
phpdoc -d ./src -t ./docs --title "Mi Proyecto" --template="clean"
```

Archivo de configuración phpdoc.xml:

```
<?xml version="1.0" encoding="UTF-8" ?>
<phpdocumentor
  configVersion="3"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="https://www.phpdoc.org"
  xsi:noNamespaceSchemaLocation="https://docs.phpdoc.org/latest/phpdoc.xsd"
>
  <title>Mi Proyecto</title>
  <paths>
    <output>docs</output>
  </paths>
  <version number="1.0.0">
    <api>
```



```

        <source dsn=".">
            <path>src</path>
        </source>
        <output>docs</output>
    </api>
</version>
</phpdocumentor>

```

Luego ejecuta simplemente:

```
phpdoc
```

2. Sami (por Symfony) Instalación - Opción A: PHAR:

```

# Descargar
curl -O http://get.sensiolabs.org/sami.phar
chmod +x sami.phar
sudo mv sami.phar /usr/local/bin/sami

```

Instalación - Opción B: Con Composer:

```
composer require --dev sami/sami
```

Archivo de configuración sami.php:

```

<?php

use Sami\Sami;
use Symfony\Component\Finder\Finder;

$iterator = Finder::create()
    ->files()
    ->name('*.php')
    ->in(__DIR__ . '/src');

return new Sami($iterator, [
    'title' => 'Mi Proyecto API',
    'build_dir' => __DIR__ . '/docs',
    'cache_dir' => __DIR__ . '/cache/sami',
    'default_opened_level' => 2,
]);

```

Generar:

```

# Con PHAR
sami update sami.php

# Con Composer
php vendor/bin/sami.php update sami.php

```

3. Doxygen Instalación:

Ubuntu/Debian

```
sudo apt-get install doxygen
```

macOS

```
brew install doxygen
```

Windows - Descargar desde <https://www.doxygen.nl/download.html>

Archivo Doxyfile básico:

```
PROJECT_NAME = "Mi Proyecto PHP"
INPUT = ./src
OUTPUT_DIRECTORY = ./docs
RECURSIVE = YES
EXTRACT_ALL = YES
GENERATE_HTML = YES
GENERATE_LATEX = NO
```

Generar:

```
doxygen Doxyfile
```

Comparación de Herramientas

Herramienta	Pros	Contras	Uso Recomendado
phpDocumentor	Estándar de facto, muy completo, templates modernos	Puede ser lento en proyectos grandes	Proyectos de cualquier tamaño
Sami	Rápido, limpio, usado por Symfony	Menos opciones de personalización	Proyectos medianos a grandes
Doxygen	Muy potente, multi-lenguaje	Interfaz menos moderna	Proyectos multi-lenguaje

Workflow Recomendado

Opción 1: Con phpDocumentor PHAR (Sin dependencias externas)

Paso 1: Descargar phpDocumentor

```
wget https://phpdoc.org/phpDocumentor.phar
chmod +x phpDocumentor.phar
```

Paso 2: Crear configuración Crea phpdoc.dist.xml en la raíz del proyecto:

```
<?xml version="1.0" encoding="UTF-8" ?>
<phpdocumentor>
  <title>Mi Proyecto</title>
  <paths>
    <output>docs/api</output>
    <cache>.phpdoc/cache</cache>
  </paths>
  <version number="latest">
    <api>
      <source dsn=".">
        <path>src</path>
      </source>
      <output>docs/api</output>
      <ignore>tests/*</ignore>
      <ignore>vendor/*</ignore>
    </api>
  </version>
</phpdocumentor>
```

Paso 3: Generar documentación

```
./phpDocumentor.phar
```

Paso 4: Visualizar

```
# Abrir directamente en el navegador
xdg-open docs/api/index.html # Linux
open docs/api/index.html # macOS
start docs/api/index.html # Windows

# O iniciar un servidor web simple con PHP
php -S localhost:8000 -t docs/api
# Abre http://localhost:8000
```

Opción 2: Con Composer (si ya lo usas en tu proyecto) Paso 1: Instalar phpDocumentor

```
composer require --dev phpdocumentor/phpdocumentor
```

Paso 2: Crear configuración Usa el mismo archivo `phpdoc.dist.xml` del ejemplo anterior.

Paso 3: Añadir scripts a `composer.json` (opcional)

```
{
    "scripts": {
        "docs": "phpdoc",
        "docs:serve": "php -S localhost:8000 -t docs/api"
    }
}
```

Paso 4: Generar y visualizar

```
# Generar documentación
composer docs
# O directamente: php vendor/bin/phpdoc

# Ver en el navegador
composer docs:serve
# Abre http://localhost:8000
```

Opción 3: Con Docker (si lo usas) Paso 1: Crear script auxiliar Crea un archivo `generate-docs.sh`:

```
#!/bin/bash
docker run --rm -v $(pwd):/data phpdoc/phpdoc:latest \
    -d /data/src \
    -t /data/docs/api \
    --title "Mi Proyecto"
```

Paso 2: Dar permisos y ejecutar

```
chmod +x generate-docs.sh
./generate-docs.sh
```

Opción 4: Instalación global en el sistema Linux/macOS:

```
# Descargar y hacer disponible globalmente
sudo wget https://phpdoc.org/phpDocumentor.phar -O /usr/local/bin/phpdoc
sudo chmod +x /usr/local/bin/phpdoc

# Ahora puedes usar 'phpdoc' desde cualquier directorio
cd /ruta/a/tu/proyecto
phpdoc -d ./src -t ./docs
```

Windows:

```
# Descargar phpDocumentor.phar a C:\php-tools\
# Crear un archivo phpdoc.bat en el mismo directorio:
```

```
@echo off
php "C:\php-tools\phpDocumentor.phar" %*

# Agregar C:\php-tools\ al PATH del sistema
# Ahora puedes usar 'phpdoc' desde cualquier directorio
```

Ignorar archivos generados

Paso 5: Integrar en .gitignore

```
# Documentación generada
/docs/api/
/.phpdoc/
```

Automatización con CI/CD

GitHub Actions (.github/workflows/docs.yml):

```
name: Generar Documentación

on:
  push:
    branches: [ main ]

jobs:
  docs:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2

      - name: Setup PHP
        uses: shivammathur/setup-php@v2
        with:
          php-version: '8.1'

      - name: Descargar phpDocumentor
        run: |
          wget https://phpdoc.org/phpDocumentor.phar
          chmod +x phpDocumentor.phar

      - name: Generar documentación
        run: ./phpDocumentor.phar -d ./src -t ./docs/api

      - name: Deploy a GitHub Pages
        uses: peaceiris/actions-gh-pages@v3
        with:
          github_token: ${ secrets.GITHUB_TOKEN }
```

```
publish_dir: ./docs/api
```

GitLab CI (.gitlab-ci.yml):

```
pages:
  image: php:8.1
  stage: deploy
  script:
    - wget https://phpdoc.org/phpDocumentor.phar
    - chmod +x phpDocumentor.phar
    - ./phpDocumentor.phar -d ./src -t ./public
  artifacts:
    paths:
      - public
  only:
    - main
```

Verificación de Documentación

Herramientas de Análisis

PHPStan (Opcional - si usas Composer):

```
composer require --dev phpstan/phpstan
composer require --dev phpstan/extension-installer
composer require --dev phpstan/phpstan-strict-rules
```

Configurar phpstan.neon:

```
parameters:
  level: max
  paths:
    - src
  checkMissingDocCommentPhpDoc: true
```

Ejecutar:

```
vendor/bin/phpstan analyse
```

Alternativa manual: Revisa manualmente cada archivo PHP y verifica que tenga bloques PHPDoc apropiados en clases, métodos y propiedades públicas.

Script de verificación básico

Puedes crear un script simple para verificar que tus archivos tengan documentación:

```
<?php
// verificar-docs.php

$directorio = 'src';
```

```

$archivos_sin_docs = [];

$iterator = new RecursiveIteratorIterator(
    new RecursiveDirectoryIterator($directorio)
);

foreach ($iterator as $archivo) {
    if ($archivo->getExtension() !== 'php') {
        continue;
    }

    $contenido = file_get_contents($archivo->getPathname());

    // Buscar clases sin PHPDoc
    if (preg_match('/^\s*class\s+\w+/m', $contenido)) {
        if (!preg_match('/\s*\*\s*\[s\S]*?\s*\s*\s*class\s+\w+/m', $contenido)) {
            $archivos_sin_docs[] = $archivo->getPathname();
        }
    }
}

if (empty($archivos_sin_docs)) {
    echo " Todos los archivos tienen documentación\n";
    exit(0);
} else {
    echo " Archivos sin documentación:\n";
    foreach ($archivos_sin_docs as $archivo) {
        echo " - $archivo\n";
    }
    exit(1);
}

```

Ejecutar:

php verificar-docs.php

Ejemplo Completo de Clase Bien Documentada

```

<?php

namespace App\Services;

use App\Models\Usuario;
use App\Exceptions\ValidationException;
use App\Exceptions\DatabaseException;
use PDO;

```

```

/**
 * Servicio de gestión de usuarios
 *
 * Proporciona funcionalidad completa para el manejo de usuarios
 * incluyendo creación, actualización, eliminación y búsqueda.
 * Implementa validación de datos y manejo de errores robusto.
 *
 * @package App\Services
 * @author Juan Pérez <juan@ejemplo.com>
 * @version 2.1.0
 * @since 1.0.0
 */
class UsuarioService
{
    /**
     * Conexión a la base de datos
     *
     * @var PDO
     */
    private $db;

    /**
     * Configuración del servicio
     *
     * @var array<string, mixed>
     */
    private $config;

    /**
     * Constructor del servicio
     *
     * @param PDO $db Conexión PDO a la base de datos
     * @param array $config Configuración opcional del servicio
     */
    public function __construct(PDO $db, array $config = [])
    {
        $this->db = $db;
        $this->config = array_merge([
            'tabla' => 'usuarios',
            'intentos_maximos' => 3,
        ], $config);
    }

    /**
     * Crea un nuevo usuario en el sistema
     *

```



```

    * Valida los datos del usuario, verifica que el email sea único,
    * hashea la contraseña usando bcrypt y almacena el registro.
    *
    * @param string $nombre Nombre completo del usuario (mínimo 3 caracteres)
    * @param string $email Dirección de correo electrónico válida y única
    * @param string $password Contraseña en texto plano (mínimo 8 caracteres)
    * @return Usuario Objeto del usuario creado con ID asignado
    * @throws ValidationException Si los datos no cumplen las validaciones
    * @throws DatabaseException Si falla la operación en base de datos
    * @see validarDatos()
    * @since 1.0.0
    */
    public function crear(string $nombre, string $email, string $password): Usuario
    {
        $this->validarDatos($nombre, $email, $password);

        if ($this->existeEmail($email)) {
            throw new ValidationException('El email ya está registrado');
        }

        $sql = "INSERT INTO {$this->config['tabla']}
            (nombre, email, password, created_at)
            VALUES (?, ?, ?, NOW())";

        try {
            $stmt = $this->db->prepare($sql);
            $stmt->execute([
                $nombre,
                $email,
                password_hash($password, PASSWORD_BCRYPT)
            ]);

            $id = $this->db->lastInsertId();
            return $this->buscarPorId($id);

        } catch (\PDOException $e) {
            throw new DatabaseException('Error al crear usuario: ' . $e->getMessage());
        }
    }

    /**
     * Busca un usuario por su ID
     *
     * @param int $id Identificador único del usuario
     * @return Usuario|null Usuario encontrado o null si no existe
     * @throws DatabaseException Si ocurre un error en la consulta

```

```

    * @since 1.0.0
    */
    public function buscarPorId(int $id): ?Usuario
    {
        $sql = "SELECT * FROM {$this->config['tabla']} WHERE id = ?";

        try {
            $stmt = $this->db->prepare($sql);
            $stmt->execute([$id]);
            $datos = $stmt->fetch(PDO::FETCH_ASSOC);

            return $datos ? new Usuario($datos) : null;

        } catch (\PDOException $e) {
            throw new DatabaseException('Error al buscar usuario: ' . $e->getMessage());
        }
    }

    /**
     * Lista usuarios con filtros y paginación
     *
     * @param array $filtros Criterios de búsqueda opcionales
     *     - 'nombre' (string): Búsqueda parcial por nombre
     *     - 'email' (string): Búsqueda exacta por email
     *     - 'activo' (bool): Filtrar por estado activo
     * @param int $limite Número máximo de resultados (por defecto: 20)
     * @param int $offset Desplazamiento para paginación (por defecto: 0)
     * @return Usuario[] Array de objetos Usuario
     * @throws DatabaseException Si ocurre un error en la consulta
     * @since 1.2.0
     */
    public function listar(array $filtros = [], int $limite = 20, int $offset = 0): array
    {
        $sql = "SELECT * FROM {$this->config['tabla']} WHERE 1=1";
        $params = [];

        if (isset($filtros['nombre'])) {
            $sql .= " AND nombre LIKE ?";
            $params[] = "%{$filtros['nombre']}%";
        }

        if (isset($filtros['email'])) {
            $sql .= " AND email = ?";
            $params[] = $filtros['email'];
        }
    }

```

```

        if (isset($filtros['activo'])) {
            $sql .= " AND activo = ?";
            $params[] = $filtros['activo'] ? 1 : 0;
        }

        $sql .= " LIMIT ? OFFSET ?";
        $params[] = $limite;
        $params[] = $offset;

        try {
            $stmt = $this->db->prepare($sql);
            $stmt->execute($params);

            $usuarios = [];
            while ($datos = $stmt->fetch(PDO::FETCH_ASSOC)) {
                $usuarios[] = new Usuario($datos);
            }

            return $usuarios;

        } catch (\PDOException $e) {
            throw new DatabaseException('Error al listar usuarios: ' . $e->getMessage());
        }
    }

    /**
     * Valida los datos de un usuario
     *
     * @param string $nombre Nombre a validar
     * @param string $email Email a validar
     * @param string $password Contraseña a validar
     * @return void
     * @throws ValidationException Si algún dato no es válido
     * @since 1.0.0
     */
    private function validarDatos(string $nombre, string $email, string $password): void
    {
        if (strlen($nombre) < 3) {
            throw new ValidationException('El nombre debe tener al menos 3 caracteres');
        }

        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new ValidationException('El email no es válido');
        }

        if (strlen($password) < 8) {

```

```

        throw new ValidationException('La contraseña debe tener al menos 8 caracteres');
    }
}

/**
 * Verifica si un email ya existe en la base de datos
 *
 * @param string $email Email a verificar
 * @return bool True si el email existe, false en caso contrario
 * @throws DatabaseException Si ocurre un error en la consulta
 * @since 1.0.0
 */
private function existeEmail(string $email): bool
{
    $sql = "SELECT COUNT(*) FROM {$this->config['tabla']} WHERE email = ?";

    try {
        $stmt = $this->db->prepare($sql);
        $stmt->execute([$email]);
        return $stmt->fetchColumn() > 0;
    } catch (\PDOException $e) {
        throw new DatabaseException('Error al verificar email: ' . $e->getMessage());
    }
}

/**
 * Actualiza los datos de un usuario existente
 *
 * @param int $id ID del usuario a actualizar
 * @param array $datos Datos a actualizar (nombre, email, activo)
 * @return Usuario Usuario actualizado
 * @throws ValidationException Si los datos no son válidos
 * @throws DatabaseException Si ocurre un error en la actualización
 * @since 1.1.0
 */
public function actualizar(int $id, array $datos): Usuario
{
    $usuario = $this->buscarPorId($id);
    if (!$usuario) {
        throw new ValidationException('Usuario no encontrado');
    }

    $campos = [];
    $params = [];

```

```

        if (isset($datos['nombre'])) {
            if (strlen($datos['nombre']) < 3) {
                throw new ValidationException('El nombre debe tener al menos 3 caracteres');
            }
            $campos[] = 'nombre = ?';
            $params[] = $datos['nombre'];
        }

        if (isset($datos['email'])) {
            if (!filter_var($datos['email'], FILTER_VALIDATE_EMAIL)) {
                throw new ValidationException('El email no es válido');
            }
            $campos[] = 'email = ?';
            $params[] = $datos['email'];
        }

        if (isset($datos['activo'])) {
            $campos[] = 'activo = ?';
            $params[] = $datos['activo'] ? 1 : 0;
        }

        if (empty($campos)) {
            return $usuario;
        }

        $campos[] = 'updated_at = NOW()';
        $params[] = $id;

        $sql = "UPDATE {$this->config['tabla']} SET " . implode(', ', $campos) . " WHERE id = " . $id;

        try {
            $stmt = $this->db->prepare($sql);
            $stmt->execute($params);
            return $this->buscarPorId($id);
        } catch (\PDOException $e) {
            throw new DatabaseException('Error al actualizar usuario: ' . $e->getMessage());
        }
    }

    /**
     * Elimina un usuario del sistema
     *
     * @param int $id ID del usuario a eliminar
     * @return bool True si se eliminó correctamente
     * @throws DatabaseException Si ocurre un error en la eliminación

```

```

    * @since 1.0.0
    */
    public function eliminar(int $id): bool
    {
        $sql = "DELETE FROM {$this->config['tabla']} WHERE id = ?";

        try {
            $stmt = $this->db->prepare($sql);
            $stmt->execute([$id]);
            return $stmt->rowCount() > 0;
        } catch (\PDOException $e) {
            throw new DatabaseException('Error al eliminar usuario: ' . $e->getMessage());
        }
    }

    /**
     * Autentica un usuario con email y contraseña
     *
     * Verifica las credenciales y controla los intentos fallidos.
     * Después de varios intentos fallidos, bloquea temporalmente la cuenta.
     *
     * @param string $email Email del usuario
     * @param string $password Contraseña en texto plano
     * @return Usuario|null Usuario autenticado o null si las credenciales son incorrectas
     * @throws ValidationException Si la cuenta está bloqueada
     * @throws DatabaseException Si ocurre un error en la consulta
     * @since 2.0.0
     */
    public function autenticar(string $email, string $password): ?Usuario
    {
        $sql = "SELECT * FROM {$this->config['tabla']} WHERE email = ?";

        try {
            $stmt = $this->db->prepare($sql);
            $stmt->execute([$email]);
            $datos = $stmt->fetch(PDO::FETCH_ASSOC);

            if (!$datos) {
                return null;
            }

            if (password_verify($password, $datos['password'])) {
                return new Usuario($datos);
            }
        }
    }

```

```

        return null;

    } catch (\PDOException $e) {
        throw new DatabaseException('Error al autenticar usuario: ' . $e->getMessage());
    }
}
}

```

Plantillas Rápidas

Plantilla para Clase

<?php

```

namespace App\NombreDelModulo;

/**
 * Descripción breve de la clase
 *
 * Descripción detallada de qué hace la clase,
 * sus responsabilidades y casos de uso.
 *
 * @package App\NombreDelModulo
 * @author Tu Nombre <tu@email.com>
 * @version 1.0.0
 * @since 1.0.0
 */
class NombreDeLaClase
{
    /**
     * Descripción de la propiedad
     *
     * @var tipo
     */
    private $propiedad;

    /**
     * Constructor de la clase
     *
     * @param tipo $parametro Descripción del parámetro
     * @throws TipoExcepcion Descripción de cuándo se lanza
     */
    public function __construct($parametro)
    {
        // Implementación
    }
}

```

```

    /**
     * Descripción del método
     *
     * @param tipo $parametro Descripción del parámetro
     * @return tipo Descripción del retorno
     * @throws TipoExcepcion Descripción de cuándo se lanza
     */
    public function metodo($parametro)
    {
        // Implementación
    }
}

```

Plantilla para Interfaz

```

<?php

namespace App\Interfaces;

/**
 * Interfaz para descripción breve
 *
 * Define el contrato para las clases que implementen
 * esta funcionalidad específica.
 *
 * @package App\Interfaces
 * @author Tu Nombre <tu@email.com>
 * @version 1.0.0
 * @since 1.0.0
 */
interface NombreDeLaInterfaz
{
    /**
     * Descripción del método
     *
     * @param tipo $parametro Descripción del parámetro
     * @return tipo Descripción del retorno
     * @throws TipoExcepcion Descripción de cuándo se lanza
     */
    public function metodo($parametro);
}

```


Plantilla para Trait

```
<?php

namespace App\Traits;

/**
 * Trait para descripción breve
 *
 * Proporciona funcionalidad común que puede ser
 * reutilizada en múltiples clases.
 *
 * @package App\Traits
 * @author Tu Nombre <tu@email.com>
 * @version 1.0.0
 * @since 1.0.0
 */
trait NombreDelTrait
{
    /**
     * Descripción del método del trait
     *
     * @param tipo $parametro Descripción del parámetro
     * @return tipo Descripción del retorno
     */
    protected function metodoDelTrait($parametro)
    {
        // Implementación
    }
}
```

Plantilla para Enumeración (PHP 8.1+)

```
<?php

namespace App\Enums;

/**
 * Enumeración para descripción breve
 *
 * Define los valores posibles para este tipo enumerado
 * y su significado en el contexto de la aplicación.
 *
 * @package App\Enums
 * @author Tu Nombre <tu@email.com>
 * @version 1.0.0
 */
```

```

    * @since 1.0.0
    */
enum EstadoUsuario: string
{
    /**
     * Usuario activo y verificado
     */
    case ACTIVO = 'activo';

    /**
     * Usuario pendiente de verificación
     */
    case PENDIENTE = 'pendiente';

    /**
     * Usuario bloqueado temporalmente
     */
    case BLOQUEADO = 'bloqueado';

    /**
     * Obtiene la descripción legible del estado
     *
     * @return string Descripción del estado
     */
    public function descripcion(): string
    {
        return match($this) {
            self::ACTIVO => 'Usuario activo',
            self::PENDIENTE => 'Pendiente de verificación',
            self::BLOQUEADO => 'Usuario bloqueado',
        };
    }
}

```

Consejos

1. Empieza desde el principio

Es más fácil documentar mientras escribes el código que hacerlo después.

2. Usa tu IDE

La mayoría de los IDEs modernos (PHPStorm, VS Code con extensiones) generan plantillas de PHPDoc automáticamente. Aprende los atajos: - **PHPStorm**: Escribe `/**` sobre una función y presiona Enter - **VS Code**: Instala “PHP DocBlocker” y usa `/**` + Enter

3. Revisa regularmente

Dedica tiempo cada semana a revisar y mejorar la documentación existente.

4. Documenta el “por qué”, no el “qué”

El código muestra QUÉ hace, la documentación debe explicar POR QUÉ lo hace de esa manera.

```
// Mal - solo repite lo que hace el código
/**
 * Multiplica el precio por 1.21
 */
public function calcularPrecioConIVA($precio) {
    return $precio * 1.21;
}

// Bien - explicar el contexto
/**
 * Calcula el precio final aplicando IVA español del 21%
 *
 * Este porcentaje es válido para bienes generales.
 * Para otros tipos de IVA, usar calcularPrecioConIVAReducido().
 *
 * @param float $precio Precio base sin impuestos
 * @return float Precio con IVA incluido
 */
public function calcularPrecioConIVA(float $precio): float {
    return $precio * 1.21;
}
```

5. Mantén consistencia

Usa el mismo estilo en todo el proyecto. Define estándares y síguelos.

6. No te excedas

No documentes lo obvio. Encuentra el balance entre muy poco y demasiado.

```
// Documentación excesiva
/**
 * Obtiene el nombre
 *
 * Este método retorna el valor de la propiedad nombre
 * que fue establecida previamente en el constructor o
 * mediante el método setNombre().
 *
 * @return string El nombre que está almacenado en la propiedad
```

```

    */
    public function getNombre(): string {
        return $this->nombre;
    }

    // Documentación adecuada
    /**
     * @return string
     */
    public function getNombre(): string {
        return $this->nombre;
    }

```

7. Actualiza con cada cambio

Código sin documentación es mejor que código con documentación incorrecta.

ULTIMAS RECOMENDACIONES

QUÉ HACER:

- Documentar todas las clases y métodos públicos
- Usar tipos específicos en lugar de mixed
- Explicar parámetros y valores de retorno
- Documentar excepciones con @throws
- Mantener la documentación actualizada
- Usar descripciones claras y concisas
- Generar documentación regularmente

QUÉ NO HACER:

- Dejar código público sin documentar
- Documentar solo por documentar
- Copiar y pegar documentación sin adaptar
- Usar tipos genéricos cuando puedes ser específico
- Dejar documentación desactualizada
- Escribir descripciones obvias o redundantes
- Olvidar las excepciones que puede lanzar un método

RECORDAR: La documentación es una inversión. Puede parecer que pierdes tiempo ahora, pero te ahorrará horas (o días) en el futuro cuando tú u otros desarrolladores necesiten entender o modificar el código.

Recursos Adicionales

Enlaces útiles

- **PSR-5 (draft) - PHPDoc Standard:** <https://github.com/php-fig/fig-standards/blob/master/proposed/phpdoc.md>
- **phpDocumentor - Documentación oficial:** <https://www.phpdoc.org/>
- **PHPDoc Tags Reference:** <https://docs.phpdoc.org/latest/guide/references/phpdoc/index.html>
- **PHP The Right Way:** <https://phptherightway.com/#documentation>
- **Doxygen Manual:** <https://www.doxygen.nl/manual/>
- **Sami Documentation:** <https://github.com/FriendsOfPHP/Sami>

Extensiones recomendadas para VS Code

- **PHP DocBlocker** - Genera bloques PHPDoc automáticamente
- **PHP Intelephense** - Mejor autocompletado y análisis
- **phpcs** - PHP Code Sniffer para verificar estándares

Extensiones para PHPStorm

PHPStorm ya incluye soporte nativo excelente para PHPDoc, pero puedes mejorarlo con: - **PHP Inspections (EA Extended)** - Análisis adicional de código
- Habilita inspecciones para “Missing PHPDoc comment”